Guide de l'administrateur

Guide de l'administrateur ⚙

Pour l'opérateur de la plateforme SaaS : créer les établissements abonnés, distribuer l'app, régler la plateforme. Droits Odoo : « Administrateur » (base.group_system).

1. Créer un établissement abonné

Configuration → Nouvel établissement (assistant) : nom de l'établissement, premier site, ses files (avec canaux distant/RDV) et, en option, son responsable. L'assistant crée la société, le site (QR d'entrée inclus), les files, un premier guichet et l'utilisateur responsable **limité à sa société** — puis ouvre la fiche du site.

Après l'assistant :

- fiche du responsable → définir son mot de passe ou lui envoyer une invitation ;
- créer les comptes **agents** (Paramètres → Utilisateurs : groupe « Agent de guichet » + société de l'établissement uniquement).

⚠ **Isolation multi-abonnés** : elle repose sur les sociétés autorisées de chaque utilisateur. Ne rattachez jamais un utilisateur d'un établissement à la société d'un autre.

2. Distribuer l'application mobile

L'application s'appelle **Kvla File** et se distribue par le catalogue commun des applications : page publique `/apps/kvla-file` (APK Android, version navigateur `/queue/u`, guide du visiteur en page et en PDF).

1. Construisez l'APK : `sh build_prod_apk.sh` (il refuse de signer sans la clé de production, et vérifie la signature et l'adresse du serveur).
2. **Configuration → App mobile** → Nouveau : version (ex. `1.5.0+7`), fichier APK, notes de version → **Publier**.

Effets immédiats : `/apps/kvla-file` sert cette version ; les applications déjà installées affichent une **bannière de mise à jour** ; le QR « installez l'app » des bornes et écrans de salle mène à la page (il est affiché même sans APK, la version navigateur existant toujours). L'ancienne adresse `/queue/app`, imprimée sur des affiches, redirige vers le catalogue. Le jour du passage au Play Store : renseignez « URL du store » dans les Paramètres — `/queue/app` y redirige, **aucun QR imprimé à refaire**.

3. Réglages de la plateforme

Configuration → Paramètres :

Réglage	Défaut	Effet
URL du store	—	Redirection de `/queue/app` vers le store
Tickets actifs max par client	5	Plafond anti-abus global
Durée de session mobile	90 j	Reconnexion par code au-delà
Notification « bientôt »	position 2	Seuil du push « Bientôt votre tour »
Expiration des RDV	60 min	RDV non enregistré → Absent (client notifié)
Auto-absent des appelés	10 min	Appelé sans réponse → Absent, guichet libéré (0 = off)

4. Prérequis techniques

- **Email sortant (indispensable au login mobile)** : configurez le serveur SMTP (Paramètres techniques → Serveurs de messagerie sortants) **et** le paramètre système `mail.default.from` (adresse cohérente avec le `from_filter` du serveur). Sans cela, l'API répond proprement « L'email n'a pas pu être envoyé » et personne ne peut se connecter.
- **URL de base** : `web.base.url` doit être l'URL publique (HTTPS en production) — elle est encodée dans tous les QR. Figez-la avec `web.base.url.freeze = True`.
- **Notifications push** : module `push_notification_hub` + compte de service Firebase (`fcm_credentials_path`). Sans FCM l'app fonctionne en mode sondage.
- **APK de production** : build signé (keystore), pointant l'URL HTTPS via `--dart-define=QUEUE_BASE_URL=…` — le HTTP en clair est refusé en release.

5. Secteurs & modèles de services

Configuration → Secteurs & modèles : catalogue de secteurs (Santé, Administration, Banque, Transport, Télécom, Commerce) et de leurs services types, avec réglages par défaut (distant, RDV, tarif). Ils pré-remplissent l'assistant « Nouvel établissement » et le bouton « Ajouter des services (modèle) » d'un site. Éditez-les librement — ce ne sont que des points de départ.

6. Paiement

Un service peut être **payant** (case « Paiement requis » + tarif). Chaque ticket naît alors « paiement en attente ». Deux règlements : l'agent **encaisse** au guichet (espèces), ou le client paie par **mobile money** depuis l'app. **Trois moyens** côté client : **Wave, au guichet, preuve de paiement** — tous soumis à **validation d'un agent** (Opérations → Paiements à valider), sauf le paiement **Wave direct** si l'API Wave est configurée.

Configuration Wave (Paramètres → File d'attente, section « Paiement — Wave ») :

- *Lien de paiement Wave (cette compagnie)* : le lien marchand Wave de la compagnie (ex. `https://pay.wave.com/m/M_ci_xxxx/c/ci/`). L'app l'ouvre avec le montant ajouté (`?amount=…`) — chaque compagnie reçoit ainsi ses propres paiements. **Vide → lien par défaut** de la plateforme.
- *Lien Wave par défaut (plateforme)* : utilisé pour les compagnies sans lien propre.
- *Clé API Wave (Checkout) + Secret du webhook* : optionnels — si renseignés, le paiement Wave devient **direct** (session API + webhook signé → payé **sans** validation guichet ; URL du webhook : `https://VOTRE-DOMAINE/api/queue/wave/webhook`).

⚠ Un **lien** de paiement Wave ne renvoie pas de confirmation automatique : le client paie via le lien, puis un **agent valide** au guichet (il vérifie que le compte Wave marchand a été crédité). Seule l'**API Checkout** (clé + secret) confirme sans validation — elle nécessite un compte marchand Wave et n'a pas encore été éprouvée en production. Aucun blocage du service en cas d'impayé.

7. Données de démonstration

Installé avec la démo, le module crée deux établissements complets (Hôpital Central, Mairie de Cocody) : services avec canaux variés (distant, RDV), guichets, historique du jour, clients mobiles (Awa Koné, Moussa Traoré), rendez-vous à venir et un compte agent — **aya.brou@hopital.demo** — déjà connecté au Guichet 1 (définissez son mot de passe pour tester « Ma console »).

8. Mécanismes automatiques (crons)

Cron	Fréquence	Rôle
Rendez-vous non honorés	15 min	RDV expirés → Absent + notification
Appelés sans réponse	2 min	Auto-absent (délai configurable) — débloque les guichets
Purge des comptes jamais vérifiés	1 j	Comptes email sans connexion > 30 j
Purge des compteurs de rate-limit	1 j	Nettoyage des compteurs anti-abus

9. Sécurité — ce qui est déjà en place

Jetons de session **hachés** (un dump de base ne donne aucune session), codes OTP hachés avec verrouillage après 5 échecs, **rate-limit par IP** (demandes de code, vérifications, téléchargements APK) fiable en multi-workers, en-têtes CSP sur toutes les pages publiques, preuve de présence (QR du site) exigée à la prise de ticket, quotas par client. Points de vigilance restants : servir la plateforme en HTTPS et surveiller la délivrabilité email.